

PROJECT-1 Report

PASSWORD STRENGTH CHECKER

Name: Harsh Nankani

Batch: September - October 2026

Organization: DecodeLabs

1. Introduction

Password Strength Checker is a cybersecurity project written in Python that checks the password strength.

The program checks the length of the password, the presence of upper case letters, numbers and symbols. Based on these conditions the password is categorized as Weak, Medium or Strong.

The project covers basic cybersecurity concepts like string handling, data validation, conditional logic, and password security. These are the key skills as stated in the project requirements.

2. Objective

The main aim of this project is to develop a program that:

- Accepts a password.
- Checks the length of its.
- Checks uppercase letters.
- Check for numbers
- Check symbols.
- Shows password strength (Weak, Medium, Strong).

3. Needs

The program should verify the following:

- Verify the length of the password.
- Check for capitalization.
- Check numbers.
- Look for symbols.

- The final password strength should be shown.

Basic password length requirement should fail for password with less than 8 characters.

4. Technologies Used:

Programming Language: Python

Concepts Used:

- String handling
- if-else conditions
- Boolean values
- any () function
- Character checking

5. how the project works

The app uses an Input to Output flow with a middle step in between.

Input: The user types a password.

Process: The program tests the password for a few items: -

- At least 8 characters –
- One uppercase letter
- One number
- One symbol

Output:

The program shows the result:

Weak or Average or Strong

6. Algorithm

1. Begin the program.
2. Prompt the user for a password.
3. Verify the password has 8 characters or more.
4. Test if it includes at least one uppercase letter.
5. Test if it includes at least one number.

6. Test if it includes at least one symbol.
7. Work out the password score.
8. If the length is under 8, mark it as Weak.
9. Else if all checks pass, mark it as Strong.
10. Else mark it as Average.
11. Show the password strength.
12. Stop the program.

7. Python Source Code

```
1  print("=====")
2  print("      PASSWORD STRENGTH CHECKER")
3  print("=====")
4
5  password = input("Enter your password: ")
6
7  length_ok = len(password) >= 8
8  has_upper = any(char.isupper() for char in password)
9  has_digit = any(char.isdigit() for char in password)
10 has_symbol = any(not char.isalnum() for char in password)
11
12 score = 0
13
14 if length_ok:
15     score += 1
16
17 if has_upper:
18     score += 1
19
20 if has_digit:
21     score += 1
22
23 if has_symbol:
24     score += 1
25
26 print("\nPassword Analysis:")
27 print("-----")
28 print("Length >= 8:", length_ok)
29 print("Uppercase letter:", has_upper)
30 print("Number:", has_digit)
31 print("Symbol:", has_symbol)
32
33 if not length_ok:
34     strength = "WEAK"
35 elif score == 4:
36     strength = "STRONG"
37 elif score >= 2:
38     strength = "MEDIUM"
39 else:
40     strength = "WEAK"
41
42 print("\nPassword Strength:", strength)
```

8. Output:

Test Case 1 – Weak Password

Input: harsh

Output: Password Strength: WEAK

Test Case 2 – Medium Password

Input: Harsh123

Output: Password Strength: Average

Test Case 3 – Strong Password

Input: Harsh@123

Output: Password Strength: STRONG

9. Complexity:

The program validates the characters in the password in order to check if it satisfies the requirement of having those character types.

As a result, its time complexity boils down to: $O(n)$ where n stands for the length of the password.

The material in the project mentions the validation to be linear on the complexity scale.

10. Security Concepts:

Password strength is important since passwords with longer lengths and variety of characters are tougher to crack.

Moreover, it discusses additional concepts like entropy, data on RAM, and timing attacks as additional cybersecurity factors.

All these concepts give opportunities to enhance or upgrade the basic project in future